18.3 ids

`ids` can be used to pluck all the IDs for the relation using the table's primary key.

```
Person.ids
# SELECT id FROM people
class Person < ActiveRecord::Base
  self.primary_key = "person_id"
end

Person.ids
# SELECT person_id FROM people
```

19 Existence of Objects

If you simply want to check for the existence of the object there's a method called `exists?`. This method will query the database using the same query as `find`, but instead of returning an object or collection of objects it will return either `true` or `false`.

```
Client.exists?(1)
```

The `exists?` method also takes multiple values, but the catch is that it will return `true` if any one of those records exists.

```
Client.exists?(id: [1,2,3])
# or
Client.exists?(name: ['John', 'Sergei'])
```

It's even possible to use `exists?` without any arguments on a model or a relation.

```
Client.where(first_name: 'Ryan').exists?
```

The above returns `true` if there is at least one client with the `first_name` 'Ryan' and `false`otherwise.

```
Client.exists?
```

The above returns `false` if the `clients` table is empty and `true` otherwise.

You can also use `any?` and `many?` to check for existence on a model or relation.

```
# via a model
Article.any?
Article.many?

# via a named scope
Article.recent.any?
Article.recent.many?

# via a relation
Article.where(published: true).any?
Article.where(published: true).many?

# via an association
Article.first.categories.any?
Article.first.categories.many?
```

20 Calculations

This section uses count as an example method in this preamble, but the options described apply to all sub-sections.

All calculation methods work directly on a model:

```
Client.count
# SELECT count(*) AS count_all FROM clients
```

Or on a relation:

```
Client.where(first_name: 'Ryan').count
# SELECT count(*) AS count_all FROM clients WHERE (first_name =
'Ryan')
```